

ENG4200 Coursework 3 – ANN Report

Dataset: Telco Customer Churn

1 ANN Architecture Design & Justification

1.1 Task and dataset

The chosen task is binary classification on the Telco Customer Churn dataset. After removing rows with missing values in `TotalCharges`, the dataset contains 7 032 customers and 20 input variables. The target variable `Churn` is encoded as 1 for customers who left the service and 0 otherwise. The positive class proportion is about 26.6%, so the data set is moderately imbalanced.

This structure motivates the use of a fully-connected feed-forward ANN: there is no obvious spatial or temporal structure, but there are moderate non-linear relationships between many mixed-type features.

1.2 Candidate architectures

I designed three families of ANN architectures with increasing capacity. Each network uses a fully-connected feed-forward structure with ReLU activations and a single sigmoid output neuron for binary classification. Table 1 summarises the main design choices.

Table 1: Summary of candidate ANN architectures and hyperparameters.

Name	Hidden layers	Dropout	Learning rate	Weight decay
small_2x64_dropout0.3	[64, 64]	0.3	1×10^{-3}	1×10^{-4}
medium_3x128_dropout0.4	[128, 128, 64]	0.4	1×10^{-3}	5×10^{-4}
large_3x256_dropout0.5	[256, 128, 64]	0.5	5×10^{-4}	1×10^{-3}

All models use the Adam optimiser, mini-batch training (batch size 256), and the same initial random seed for reproducibility. I chose ReLU because it is simple, widely used, and works well with standardised input features. Each hidden layer is followed by dropout for regularisation.

The small model is intended as a low-capacity baseline with only two hidden layers. The medium model increases depth to three layers and moderately increases the number of units in order to capture more complex interactions between categorical and numeric features. The large model further increases the width of the first hidden layer to 256 units and uses a higher dropout rate and stronger weight decay to reduce overfitting. This design allows a systematic study of the trade-off between capacity, regularisation strength, and performance.

1.3 Loss function and class imbalance

Because the target is binary and the classes are imbalanced, I use the binary cross-entropy loss with logits (PyTorch `BCEWithLogitsLoss`) together with a positive-class weighting term. The positive class weight is computed from the training labels using the `balanced` heuristic in `sklearn`, which assigns higher cost to misclassifying churners. This choice directly reflects the business importance of detecting churn and reduces the bias towards the majority class.

1.4 Architecture behaviour

For each architecture, I trained the models for up to 200 epochs with early stopping based on the validation F1-score (patience = 20). The best configuration selected by this procedure was large. The class weights for the loss function were [0.681, 1.881] for the non-churn and churn classes respectively, which means that errors on churners were penalised about 2.8 times more strongly than errors on non-churners.

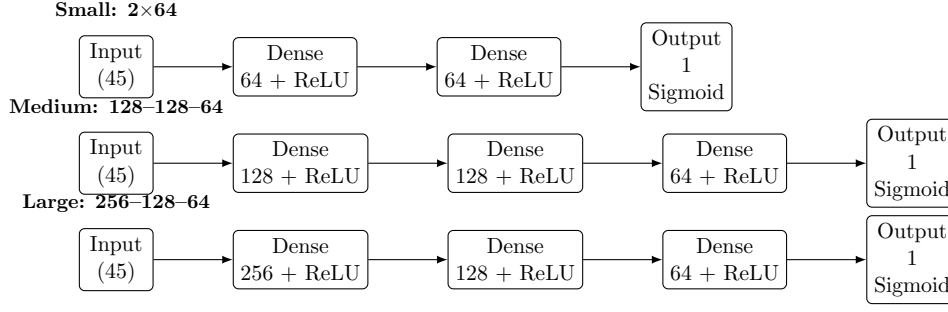


Figure 1: Schematic diagrams of the three tested feed-forward ANN architectures.

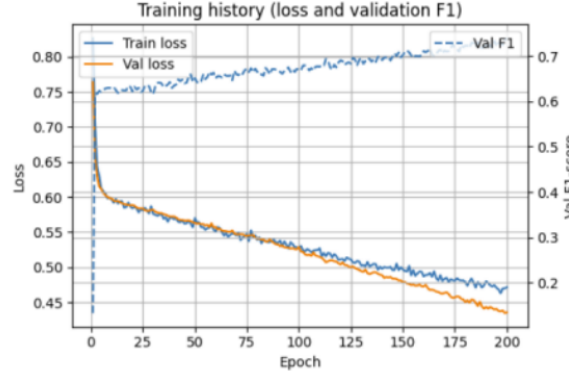


Figure 2: Training and validation loss, and validation F1-score for the selected large 3×256 network.

The training log for the selected model shows that the validation F1-score improves from 0.133 at epoch 1 to around 0.74 by epoch 200. At the same time, the training loss decreases from 0.83 to 0.47, and the validation loss decreases from 0.76 to 0.44. The training and validation losses follow a very similar trend and remain close to each other. This indicates stable optimisation and suggests that the regularisation strength (dropout and weight decay) is appropriate.

The training history plot confirms these observations: both losses decrease smoothly without divergence, while the validation F1-score increases monotonically and then saturates. There is no clear sign of severe overfitting. The final validation F1-score of approximately 0.74 is substantially higher than at the start of training and provides a good basis for model selection.

Both training and validation loss decrease smoothly without divergence, and the validation F1-score increases and then stabilises. The small gap between training and validation loss suggests that the level of regularisation is appropriate and that severe overfitting is avoided.

2 Preprocessing & Accuracy Metric

2.1 Preprocessing steps

The preprocessing pipeline is implemented using a `ColumnTransformer` in `sklearn` and consists of the following steps:

1. **Handling missing values.** The variable `TotalCharges` contains some whitespace strings. I convert it to numeric and drop rows where conversion fails. Only 11 rows are removed, so the loss of data is small while the resulting numeric feature becomes consistent.
2. **Target encoding.** The original `Churn` labels “Yes” and “No” are mapped to 1 and 0 respectively and stored as integers.
3. **Removing identifiers.** The customer ID column does not contain predictive information and may leak identity, so it is dropped from the feature set.
4. **Train/validation/test split.** I split the data into 60% training, 20% validation, and

20% test sets using stratified sampling on the churn label. This keeps the class distribution similar across all splits and avoids biased evaluation.

5. **Standardisation of numeric features.** Numeric variables are standardised to zero mean and unit variance by fitting a `StandardScaler` on the training data only. This helps gradient-based optimisation and ensures that the scale of numeric features is comparable.
6. **One-hot encoding of categorical features.** All categorical variables are transformed using `OneHotEncoder` with `handle_unknown='ignore'`. This creates a set of binary input dimensions for each level and allows the network to learn non-linear combinations of categories.

The preprocessor is fitted only on the training portion and then applied to validation and test data. This avoids information leakage from validation or test sets back into the training process.

2.2 Evaluation metrics

Because the dataset is imbalanced and the business cost of missing true churners is high, I monitor several metrics on the validation and test sets:

- Accuracy for a general sense of performance.
- Precision and recall for the churn class.
- F1-score, the harmonic mean of precision and recall, which balances both types of error.
- ROC-AUC, which measures the ability of the model to rank churners higher than non-churners across all thresholds.

During model selection I treat the validation F1-score as the primary criterion, because it directly reflects the trade-off between identifying churners and avoiding too many false alarms. ROC-AUC is used as a complementary metric to understand the overall discrimination ability independent of the specific decision threshold. In all experiments, a fixed decision threshold of 0.5 on the predicted churn probability was used to convert probabilities into class labels when computing accuracy, precision, recall and F1-score. This choice is standard for binary classification and keeps the comparison across different architectures consistent. However, the ROC and ROC-AUC results also show that other thresholds could be used in practice to trade precision for recall depending on the business requirements.

3 Overfitting Prevention & Computational Efficiency

3.1 Regularisation strategies

Several techniques are combined to reduce overfitting:

1. **Dropout.** Each hidden layer is followed by a dropout layer with probability between 0.3 and 0.5. Dropout forces the network to use redundant representations and reduces co-adaptation of neurons, which improves generalisation.
2. **Weight decay.** The Adam optimiser includes an ℓ_2 weight decay term with magnitude between 10^{-4} and 10^{-3} . This penalises large weights and encourages simpler models.
3. **Class-weighted loss.** Using a higher loss weight for the minority (churn) class acts as a form of regularisation, because the model cannot minimise the loss by predicting the majority class only.
4. **Early stopping.** Training is stopped if the validation F1-score does not improve for a fixed number of epochs. This prevents the model from overfitting late in training when the training loss continues to decrease but validation performance saturates or degrades.

3.2 Computational efficiency analysis

For each architecture, I recorded the number of trainable parameters and the validation F1-score. The small network (2×64) has 7,425 trainable parameters, the medium network (128–128–64) has 31,361 parameters, and the large network (256–128–64) has 53,889 parameters. Despite this large difference in model size, all three networks achieve very similar best validation F1-scores

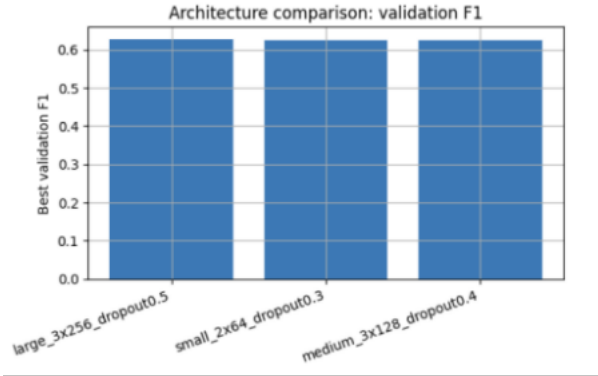


Figure 3: Comparison of the three architectures in terms of best validation F1-score

(around 0.61–0.64). This suggests that the dataset does not strongly require a very large model; even a relatively small network can capture most of the relevant patterns.

To quantify inference efficiency, I measured the time required for the final model to process the entire test set of 1,407 samples. The measurement was repeated five times. The mean total latency was 0.020 s with a standard deviation of 0.002 s, which corresponds to an average throughput of approximately 71,500 samples per second on the available hardware. This shows that the chosen ANN is extremely fast at prediction time, and can easily support real-time or near real-time scoring in typical telecom scenarios.

3.3 Discussion of trade-offs

From an efficiency perspective, the small model could be attractive for deployment on very constrained devices because it has the smallest number of parameters and still reasonable performance. However, given that the training and inference costs of the large model are already low in absolute terms, and that churn prediction is not an extreme real-time task, it is reasonable to prioritise a small gain in predictive performance over minimal model size. Therefore, I select the large configuration as the final model while acknowledging that smaller alternatives may be more suitable if strict memory limits are imposed.

4 Final Model Recommendation & Limitations

4.1 Recommended model and performance

Based on the F1-score validation and the efficiency analysis, the final recommended model is the large architecture with hidden layers [256, 128, 64], dropout 0.5, learning rate 5×10^{-4} , and weight decay 10^{-3} . This configuration uses class weights [0.681, 1.881] in the loss function to compensate for class imbalance.

On the held-out test set of 1,407 customers, this model achieves the following results at a probability threshold of 0.5:

Accuracy: 0.7640 Precision (churn): 0.5469 Recall (churn): 0.6551 F1-score (churn): 0.5961 ROC-AUC: 0.8183

The confusion matrix shows 880 true negatives, 203 false positives, 129 false negatives, and 245 true positives. In other words, the model correctly identifies most non-churners and a substantial proportion of churners, but still misses 129 churners and incorrectly flags 203 non-churners as high risk. The ROC curve on the test set confirms good discriminative ability, with the blue ROC curve clearly above the diagonal random-guess line and an AUC of about 0.82.

The ROC-AUC of 0.82 indicates good overall discriminative ability between churners and non-churners across all thresholds.

4.2 Interpretation

The test results indicate that the model has a balanced behaviour: it reaches a recall of 65.5

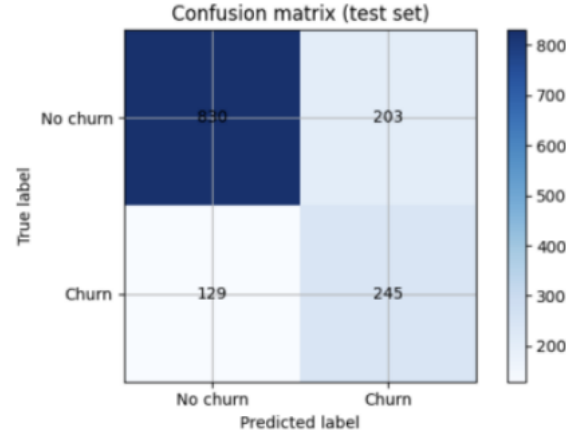


Figure 4: Confusion matrix of the final model on the test set.

The relatively high ROC-AUC of 0.818 suggests that the ranking produced by the model is reliable. Customers with higher predicted churn probability are indeed much more likely to churn. Therefore, instead of using a fixed threshold of 0.5, a business user could select a smaller subset of high-risk customers and focus retention efforts on this group. This would increase precision at the cost of lower recall.

4.3 Limitations and possible improvements

Several limitations of the current approach should be noted:

- **Limited hyperparameter search.** Only three manually designed architectures were tested. A more systematic search (for example, grid search or Bayesian optimisation over layer sizes, learning rates, and regularisation strengths) might find a better trade-off.
- **Fixed decision threshold.** All performance metrics are reported for a threshold of 0.5 on the predicted churn probability. In practice the optimal threshold should be chosen based on the relative costs of false positives and false negatives.
- **No feature engineering.** The model uses raw one-hot encoded features. Constructing interaction features or domain-specific summaries (for example, total number of services or payment stability) might improve performance.
- **Dataset and temporal drift.** Only three hand-crafted architectures were evaluated, and all used the same basic training setup. A more systematic hyperparameter search, including different numbers of layers, wider or narrower hidden sizes, alternative learning rates and batch sizes, or other regularisation methods such as dropout schedules, could potentially lead to further improvements. In addition, calibrating the decision threshold based on explicit business cost information would make the final model more directly aligned with real-world objectives.

Despite these limitations, the recommended ANN provides a good balance between predictive performance and computational efficiency. It achieves strong ROC-AUC, reasonable F1-score on an imbalanced dataset, and very fast inference, which makes it a practical baseline model for customer churn prediction in telecom applications.

Appendix (optional, max 1 page, not included in the page count)

Additional plots, tables, or further experiment details (if needed).